Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №3

По дисциплине «Модели решения задач в интеллектуальных системах»

Тема: «Бинарная классификация с использованием кросс-энтропии (логистическая регрессия)»

Выполнил:

Студент 3 курса

Группы ИИ-26

Кушнеревич Е.А

Проверила:

Андренко К.В.

Брест 2026

Цель работы: Изучить принципы бинарной классификации с вероятностной точки зрения. Реализовать однослойную нейронную сеть (логистический регрессор) с сигмоидной функцией активации и функцией потерь в виде бинарной кросс-энтропии (Binary Cross-Entropy, BCE). Провести сравнительный анализ скорости сходимости и качества классификации с моделями, использующими среднеквадратичную ошибку (ЛР №1) и адаптивный шаг обучения (ЛР №2).

Ход работы:

1. Модифицировать программу из ЛР №1:
   - Заменить пороговую функцию активации на сигмоидную.
   - Заменить функцию ошибки MSE на бинарную кросс-энтропию (BCE).
   - Реализовать вычисление градиента $(\hat{y} - e)x_i$ для обновления весов.
2. Реализовать обучение с **фиксированным** шагом (взять значение α, показавшее наилучший результат в ЛР №1).
Запоминать суммарную ошибку $E_s(p)$ на каждой эпохе p.

3. Реализовать обучение с **адаптивным** шагом:
   - На каждой итерации (после подачи одного образа) вычислять $\alpha(t)$ (формула (2.36)).
   - Использовать это $\alpha(t)$ для обновления весов и порога по градиенту BCE.
   - Сохранять значения $E_s(p)$ для построения графика.
4. Провести сравнительный анализ:
   - Построить на одном графике кривые сходимости $E_s(p)$ для четырёх конфигураций:
     - MSE + фиксированный шаг (данные из ЛР №1);
     - MSE + адаптивный шаг (данные из ЛР №2);
     - BCE + фиксированный шаг;
     - BCE + адаптивный шаг.
   - Зафиксировать число эпох, необходимое для достижения заданной точности $E_s \leq E_e$ (взять E_e из ЛР №2, например 0.01).
5. Визуализировать результаты:
   - Отобразить точки обучающей выборки (разными цветами для классов 0 и 1).
   - Построить разделяющую линию, полученную после обучения модели с BCE (адаптивный шаг).
   - Для сравнения также показать линию из ЛР №1 (если она отличается). **Описать** сравнение двух линий.
6. Реализовать режим функционирования:
   - Пользователь вводит координаты x_1, x_2.
   - Сеть вычисляет вероятность $\hat{y}$ и, применяя порог, определяет предсказанный класс.
   - Точка отображается на графике с разделяющей линией (добавляется маркером с подписью класса или вероятности).
7. Сравнить скорость сходимости BCE и MSE, объяснить различия с позиции теории. Оценить влияние адаптивного шага на обучение с BCE. Сравнить итоговые разделяющие линии, полученные разными методами, и объяснить возможные расхождения.

8. Написать вывод о применимости разных функций потерь для задачи бинарной классификации.

8.

x_1	x_2	e
5	6	0
-5	6	1
5	-6	1
-5	-6	1

Код программы:

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([
    [ 5,  6],
    [-5,  6],
    [ 5, -6],
    [-5, -6]
])

etalon_value = np.array([0, 1, 1, 1]).reshape(-1, 1).astype(float)

def sigmoid(s):
    return 1.0 / (1.0 + np.exp(-s))

def linear_output(W, X, T):
    return X @ W - T

def bce_loss(y_pred, y_true):
    eps = 1e-12
    y_pred = np.clip(y_pred, eps, 1 - eps)
    return -np.sum(y_true * np.log(y_pred) + (1 - y_true) * np.log(1 -
y_pred))

def mse_loss(y_pred, y_true):
    return np.sum((y_pred - y_true) ** 2)

def bce_fixed_fit(X, y, alpha=0.01, Ee=0.01, max_epochs=20000):
    N, n = X.shape
    W = np.zeros((n, 1))
    T = 0.0
    Es_hist = []
    stopped_epoch = max_epochs

    print(f"(BCE + фиксированный) alpha={alpha}, Ee={Ee}")

    for ep in range(max_epochs):
        order = np.random.permutation(N)
        for idx in order:
```

```
            xi = X[idx:idx+1]
            ti = float(y[idx, 0])

            s   = float(linear_output(W, xi, T)[0, 0])
            yi  = sigmoid(s)
            d   = yi - ti

            W  -= alpha * xi.T * d
            T  += alpha * d

        preds = sigmoid(linear_output(W, X, T))
        Es = bce_loss(preds, y)
        Es_hist.append(Es)

        if ep % 200 == 0 and ep > 0:
            print(f"  эпоха {ep:5d} | Es = {Es:.6f}")

        if Es <= Ee:
            stopped_epoch = ep + 1
            print(f"  Остановка на эпохе {stopped_epoch}, Es={Es:.6f}\n")
            break
    else:
        print(f"  Лимит {max_epochs} эпох, Es={Es_hist[-1]:.6f}\n")

    return W, T, Es_hist, stopped_epoch

def bce_adaptive_fit(X, y, Ee=0.01, max_epochs=20000):
    N, n = X.shape
    W = np.zeros((n, 1))
    T = 0.0
    Es_hist = []
    stopped_epoch = max_epochs

    print(f"(BCE + адаптивный) α(t), Ee={Ee}")

    for ep in range(max_epochs):
        order = np.random.permutation(N)
        for idx in order:
            xi      = X[idx:idx+1]
            ti      = float(y[idx, 0])
            norm_sq = float(np.sum(xi ** 2))
            alpha_t = 1.0 / norm_sq if norm_sq > 1e-12 else 0.0

            s   = float(linear_output(W, xi, T)[0, 0])
            yi  = sigmoid(s)
            d   = yi - ti

            W  -= alpha_t * xi.T * d
            T  += alpha_t * d
```

```
        preds = sigmoid(linear_output(W, X, T))
        Es = bce_loss(preds, y)
        Es_hist.append(Es)

        if ep % 50 == 0 and ep > 0:
            print(f"  эпоха {ep:5d} | Es = {Es:.6f}")

        if Es <= Ee:
            stopped_epoch = ep + 1
            print(f"  Остановка на эпохе {stopped_epoch}, Es={Es:.6f}\n")
            break
    else:
        print(f"  Лимит {max_epochs} эпох, Es={Es_hist[-1]:.6f}\n")

    return W, T, Es_hist, stopped_epoch

def mse_fixed_fit(X, y, alpha=0.01, Ee=0.5, max_epochs=20000):
    N, n = X.shape
    W = np.zeros((n, 1))
    T = 0.0
    Es_hist = []
    stopped_epoch = max_epochs
    for ep in range(max_epochs):
        order = np.random.permutation(N)
        for idx in order:
            xi = X[idx:idx+1]; ti = float(y[idx, 0])
            s  = float(linear_output(W, xi, T)[0, 0])
            d  = s - ti
            W -= alpha * xi.T * d; T += alpha * d
        Es = mse_loss(linear_output(W, X, T), y)
        Es_hist.append(Es)
        if Es <= Ee:
            stopped_epoch = ep + 1; break
    return W, T, Es_hist, stopped_epoch

def mse_adaptive_fit(X, y, Ee=0.5, max_epochs=20000):
    N, n = X.shape
    W = np.zeros((n, 1))
    T = 0.0
    Es_hist = []
    stopped_epoch = max_epochs
    for ep in range(max_epochs):
        order = np.random.permutation(N)
        for idx in order:
            xi = X[idx:idx+1]; ti = float(y[idx, 0])
            norm_sq = float(np.sum(xi ** 2))
            alpha_t = 1.0 / norm_sq if norm_sq > 1e-12 else 0.0
            s  = float(linear_output(W, xi, T)[0, 0])
```

```
            d  = s - ti
            W -= alpha_t * xi.T * d; T += alpha_t * d
        Es = mse_loss(linear_output(W, X, T), y)
        Es_hist.append(Es)
        if Es <= Ee:
            stopped_epoch = ep + 1; break
    return W, T, Es_hist, stopped_epoch

np.random.seed(42)
Ee_MSE = 0.5
Ee_BCE = 0.01

W_mf,  T_mf,  es_mf,  ep_mf  = mse_fixed_fit (x, etalon_value, alpha=0.01,
Ee=Ee_MSE)
np.random.seed(42)
W_ma,  T_ma,  es_ma,  ep_ma  = mse_adaptive_fit (x, etalon_value, Ee=Ee_MSE)
np.random.seed(42)
W_bf,  T_bf,  es_bf,  ep_bf  = bce_fixed_fit (x, etalon_value, alpha=0.01,
Ee=Ee_BCE)
np.random.seed(42)
W_ba,  T_ba,  es_ba,  ep_ba  = bce_adaptive_fit (x, etalon_value, Ee=Ee_BCE)

print("\n Итог: число эпох до Es ≤ Ee ")
print(f"  MSE + фиксированный : {ep_mf}")
print(f"  MSE + адаптивный    : {ep_ma}")
print(f"  BCE + фиксированный : {ep_bf}")
print(f"  BCE + адаптивный    : {ep_ba}")

fig, axes = plt.subplots(1, 2, figsize=(16, 6))
ax1 = axes[0]

ax1.plot(es_mf, lw=2, color="#2563EB",
         label=f"MSE + фиксированный α=0.01  (эпох: {ep_mf})")
ax1.plot(es_ma, lw=2, color="#60A5FA",
         label=f"MSE + адаптивный α(t)        (эпох: {ep_ma})")
ax1.plot(es_bf, lw=2, color="#DC2626",
         label=f"BCE + фиксированный α=0.01  (эпох: {ep_bf})")
ax1.plot(es_ba, lw=2, color="#F97316",
         label=f"BCE + адаптивный α(t)        (эпох: {ep_ba})")
ax1.axhline(Ee_BCE, color="gray", lw=1.4, ls="--", label=f"Eₑ(BCE) =
{Ee_BCE}")
ax1.axhline(Ee_MSE,  color="#3B82F6", lw=1.2, ls=":", label=f"Eₑ(MSE) =
{Ee_MSE}")
ax1.set_yscale("log")
ax1.set_xlabel("Эпоха (p)", fontsize=12)
ax1.set_ylabel("Es(p)", fontsize=12)
ax1.set_title("Кривые сходимости: все 4 конфигурации", fontsize=13,
fontweight='bold')
ax1.legend(fontsize=9); ax1.grid(alpha=0.35)
```

```
ax2 = axes[1]

cls0 = etalon_value[:, 0] == 0
cls1 = etalon_value[:, 0] == 1

ax2.scatter(x[cls0][:, 0], x[cls0][:, 1], s=200, color="#3B82F6",
            edgecolors="black", lw=1.5, label="Класс 0", zorder=5)
ax2.scatter(x[cls1][:, 0], x[cls1][:, 1], s=200, color="#EF4444",
            edgecolors="black", lw=1.5, label="Класс 1", zorder=5)

for pt, lbl in zip(x, ["(5,6)", "(-5,6)", "(5,-6)", "(-5,-6)"]):
    ax2.annotate(lbl, xy=(pt[0], pt[1]), xytext=(pt[0]+0.3, pt[1]+0.6),
fontsize=9)

xs_line = np.linspace(-9, 9, 500)

def draw_boundary(ax, W, T, color, label, threshold=0.0):
    w1, w2 = W.flatten()
    if abs(w2) > 1e-9:
        ys = (T + threshold - w1 * xs_line) / w2
        ax.plot(xs_line, ys, color=color, lw=2.5, label=label)

draw_boundary(ax2, W_mf, T_mf, "#2563EB", f"MSE+фикс  (Эпох: {ep_mf})",
threshold=0.5)
draw_boundary(ax2, W_ma, T_ma, "#60A5FA", f"MSE+адап  (Эпох: {ep_ma})",
threshold=0.5)
draw_boundary(ax2, W_bf, T_bf, "#DC2626", f"BCE+фикс  (Эпох: {ep_bf})",
threshold=0.0)
draw_boundary(ax2, W_ba, T_ba, "#F97316", f"BCE+адап  (Эпох: {ep_ba})",
threshold=0.0)

ax2.set_xlim(-9, 9); ax2.set_ylim(-9, 9)
ax2.set_xlabel("x_1", fontsize=12); ax2.set_ylabel("x_2", fontsize=12)
ax2.set_title("Разделяющие линии: все 4 конфигурации", fontsize=13,
fontweight='bold')
ax2.legend(fontsize=9); ax2.grid(alpha=0.3)
ax2.axhline(0, color='black', lw=0.5); ax2.axvline(0, color='black', lw=0.5)

plt.tight_layout()
plt.show()

added_pts, added_cls = [], []

print("−" * 50)
print(" Режим функционирования (BCE + адаптивный)")
print(" Введите координаты x_1 x_2, или 'exit'\n")

while True:
```

```
    user_input = input(" -> ").strip()
    if user_input in ("exit", ""):
        break
    try:
        x1_val, x2_val = map(float, user_input.split())
        point = np.array([[x1_val, x2_val]])

        s    = float(linear_output(W_ba, point, T_ba)[0, 0])
        prob = sigmoid(s)
        cls  = 1 if prob >= 0.5 else 0

        print(f"  S = {s:+.6f}")
        print(f"  ŷ = P(класс=1) = {prob:.6f}")
        print(f"  Класс = {cls} ({'красный' if cls else 'синий'})\n")

        added_pts.append([x1_val, x2_val])
        added_cls.append(cls)

        fig, ax = plt.subplots(figsize=(8, 8))
        ax.scatter(x[cls0][:, 0], x[cls0][:, 1], s=200, color="#3B82F6",
                   edgecolors="black", label="Класс 0")
        ax.scatter(x[cls1][:, 0], x[cls1][:, 1], s=200, color="#EF4444",
                   edgecolors="black", label="Класс 1")

        draw_boundary(ax, W_ba, T_ba, "#F97316", "BCE+адап (граница)",
threshold=0.0)
        draw_boundary(ax, W_ma, T_ma, "#60A5FA", "MSE+адап (для сравнения)",
threshold=0.5)

        for p, c in zip(added_pts, added_cls):
            ax.scatter(p[0], p[1], marker="X", s=280,
                       color="#EF4444" if c else "#3B82F6",
                       edgecolors="black", zorder=6)

        ax.set_xlim(-9, 9); ax.set_ylim(-9, 9)
        ax.set_xlabel("x₁"); ax.set_ylabel("x₂")
        ax.set_title(f"Точка ({x1_val}, {x2_val}) -> ŷ={prob:.3f} -> класс
{cls}")
        ax.legend(); ax.grid(alpha=0.3)
        ax.axhline(0, color='black', lw=0.5); ax.axvline(0, color='black',
lw=0.5)
        plt.tight_layout(); plt.show()

    except Exception:
        print("  Ошибка ввода. Формат: x_1 x_2\n")

print("Работа завершена.")
```

Результат тестирования:

(BCE + фиксированный) alpha=0.01, Ee=0.01

эпоха 200 | Es = 0.473885

эпоха 400 | Es = 0.264953

эпоха 600 | Es = 0.181297

эпоха 800 | Es = 0.137101

эпоха 1000 | Es = 0.109976

эпоха 1200 | Es = 0.091701

эпоха 1400 | Es = 0.078574

эпоха 1600 | Es = 0.068703

эпоха 1800 | Es = 0.061016

эпоха 2000 | Es = 0.054863

эпоха 2200 | Es = 0.049829

эпоха 2400 | Es = 0.045636

эпоха 2600 | Es = 0.042089

эпоха 2800 | Es = 0.039051

эпоха 3000 | Es = 0.036420

эпоха 3200 | Es = 0.034119

эпоха 3400 | Es = 0.032090

эпоха 3600 | Es = 0.030288

эпоха 3800 | Es = 0.028677

эпоха 4000 | Es = 0.027227

эпоха 4200 | Es = 0.025917

эпоха 4400 | Es = 0.024726

эпоха 4600 | Es = 0.023640

эпоха 4800 | Es = 0.022645

эпоха 5000 | Es = 0.021730

эпоха 5200 | Es = 0.020886

эпоха 5400 | Es = 0.020104

эпоха 5600 | Es = 0.019379

эпоха 5800 | Es = 0.018705

эпоха 6000 | Es = 0.018075

эпоха 6200 | Es = 0.017487

эпоха 6400 | Es = 0.016935

эпоха 6600 | Es = 0.016417

эпоха 6800 | Es = 0.015930

эпоха 7000 | Es = 0.015470

эпоха 7200 | Es = 0.015037

эпоха 7400 | Es = 0.014627

эпоха 7600 | Es = 0.014239

эпоха 7800 | Es = 0.013870

эпоха 8000 | Es = 0.013521

эпоха 8200 | Es = 0.013188

эпоха 8400 | Es = 0.012872

эпоха 8600 | Es = 0.012570

эпоха 8800 | Es = 0.012282

эпоха 9000 | Es = 0.012007

эпоха 9200 | Es = 0.011744

эпоха 9400 | Es = 0.011492

эпоха 9600 | Es = 0.011250

эпоха 9800 | Es = 0.011019

эпоха 10000 | Es = 0.010797

эпоха 10200 | Es = 0.010584

эпоха 10400 | Es = 0.010379

эпоха 10600 | Es = 0.010181

Остановка на эпохе 10792, Es=0.010000

(BCE + адаптивный) α(t), Ee=0.01

эпоха 50 | Es = 0.802297

эпоха 100 | Es = 0.531614

эпоха 150 | Es = 0.389854

эпоха 200 | Es = 0.306107

эпоха 250 | Es = 0.251075

эпоха 300 | Es = 0.212439

эпоха 350 | Es = 0.183887

эпоха 400 | Es = 0.161974

эпоха 450 | Es = 0.144653

эпоха 500 | Es = 0.130625

эпоха 550 | Es = 0.119043

эпоха 600 | Es = 0.109328

эпоха 650 | Es = 0.101055

эпоха 700 | Es = 0.093937

эпоха 750 | Es = 0.087746

эпоха 800 | Es = 0.082314

эпоха 850 | Es = 0.077508

эпоха 900 | Es = 0.073228

эпоха 950 | Es = 0.069393

эпоха 1000 | Es = 0.065937

эпоха 1050 | Es = 0.062806

эпоха 1100 | Es = 0.059957

эпоха 1150 | Es = 0.057353

эпоха 1200 | Es = 0.054967

эпоха 1250 | Es = 0.052767

эпоха 1300 | Es = 0.050737

эпоха 1350 | Es = 0.048856

эпоха 1400 | Es = 0.047109

эпоха 1450 | Es = 0.045482

эпоха 1500 | Es = 0.043963

эпоха 1550 | Es = 0.042542

эпоха 1600 | Es = 0.041210

эпоха 1650 | Es = 0.039958

эпоха 1700 | Es = 0.038779

эпоха 1750 | Es = 0.037668

эпоха 1800 | Es = 0.036618

эпоха 1850 | Es = 0.035625

эпоха 1900 | Es = 0.034684

эпоха 1950 | Es = 0.033792

эпоха 2000 | Es = 0.032944

эпоха 2050 | Es = 0.032137

эпоха 2100 | Es = 0.031369

эпоха 2150 | Es = 0.030637

эпоха 2200 | Es = 0.029938

эпоха 2250 | Es = 0.029270

эпоха 2300 | Es = 0.028631

эпоха 2350 | Es = 0.028019

эпоха 2400 | Es = 0.027433

эпоха 2450 | Es = 0.026870

эпоха 2500 | Es = 0.026331

эпоха 2550 | Es = 0.025812

эпоха 2600 | Es = 0.025314

эпоха 2650 | Es = 0.024834

эпоха 2700 | Es = 0.024372

эпоха 2750 | Es = 0.023927

эпоха 2800 | Es = 0.023498

эпоха 2850 | Es = 0.023083

эпоха 2900 | Es = 0.022684

эпоха 2950 | Es = 0.022297

эпоха 3000 | Es = 0.021924

эпоха 3050 | Es = 0.021563

эпоха 3100 | Es = 0.021214

эпоха 3150 | Es = 0.020875

эпоха 3200 | Es = 0.020548

эпоха 3250 | Es = 0.020230

эпоха 3300 | Es = 0.019922

эпоха 3350 | Es = 0.019623

эпоха 3400 | Es = 0.019333

эпоха 3450 | Es = 0.019052

эпоха 3500 | Es = 0.018779

эпоха 3550 | Es = 0.018513

эпоха 3600 | Es = 0.018255

эпоха 3650 | Es = 0.018003

эпоха 3700 | Es = 0.017759

эпоха 3750 | Es = 0.017521

эпоха 3800 | Es = 0.017289

эпоха 3850 | Es = 0.017064

эпоха 3900 | Es = 0.016844

эпоха 3950 | Es = 0.016630

эпоха 4000 | Es = 0.016421

эпоха 4050 | Es = 0.016217

эпоха 4100 | Es = 0.016019

эпоха 4150 | Es = 0.015825

эпоха 4200 | Es = 0.015636

эпоха 4250 | Es = 0.015451

эпоха 4300 | Es = 0.015270

эпоха 4350 | Es = 0.015094

эпоха 4400 | Es = 0.014922

эпоха 4450 | Es = 0.014753

эпоха 4500 | Es = 0.014589

эпоха 4550 | Es = 0.014428

эпоха 4600 | Es = 0.014270

эпоха 4650 | Es = 0.014116

эпоха 4700 | Es = 0.013965

эпоха 4750 | Es = 0.013818

эпоха 4800 | Es = 0.013673

эпоха 4850 | Es = 0.013531

эпоха 4900 | Es = 0.013393

эпоха 4950 | Es = 0.013257

эпоха 5000 | Es = 0.013124

эпоха 5050 | Es = 0.012993

эпоха 5100 | Es = 0.012865

эпоха 5150 | Es = 0.012740

эпоха 5200 | Es = 0.012617

эпоха 5250 | Es = 0.012496

эпоха 5300 | Es = 0.012378

эпоха 5350 | Es = 0.012262

эпоха 5400 | Es = 0.012148

эпоха 5450 | Es = 0.012036

эпоха 5500 | Es = 0.011926

эпоха 5550 | Es = 0.011818

эпоха 5600 | Es = 0.011712

эпоха 5650 | Es = 0.011608

эпоха 5700 | Es = 0.011506

эпоха 5750 | Es = 0.011405

эпоха 5800 | Es = 0.011306

эпоха 5850 | Es = 0.011209

эпоха 5900 | Es = 0.011114

эпоха 5950 | Es = 0.011020

эпоха 6000 | Es = 0.010928

эпоха 6050 | Es = 0.010837

эпоха 6100 | Es = 0.010748

эпоха 6150 | Es = 0.010660

эпоха 6200 | Es = 0.010574

эпоха 6250 | Es = 0.010489

эпоха 6300 | Es = 0.010406

эпоха 6350 | Es = 0.010323

эпоха 6400 | Es = 0.010242

эпоха 6450 | Es = 0.010163

эпоха 6500 | Es = 0.010084

эпоха 6550 | Es = 0.010007

Остановка на эпохе 6556, Es=0.009999

Итог: число эпох до Es ≤ Ee

MSE + фиксированный: 24

MSE + адаптивный: 21

BCE + фиксированный: 10792

BCE + адаптивный: 6556

Режим функционирования (BCE + адаптивный)

Введите координаты x_1 x_2, или 'exit'

-> 5 6

S = -8.057577

ŷ = P(класс=1) = 0.000317

Класс = 0 (синий)

График кривой сходимости для четырех конфигурации:

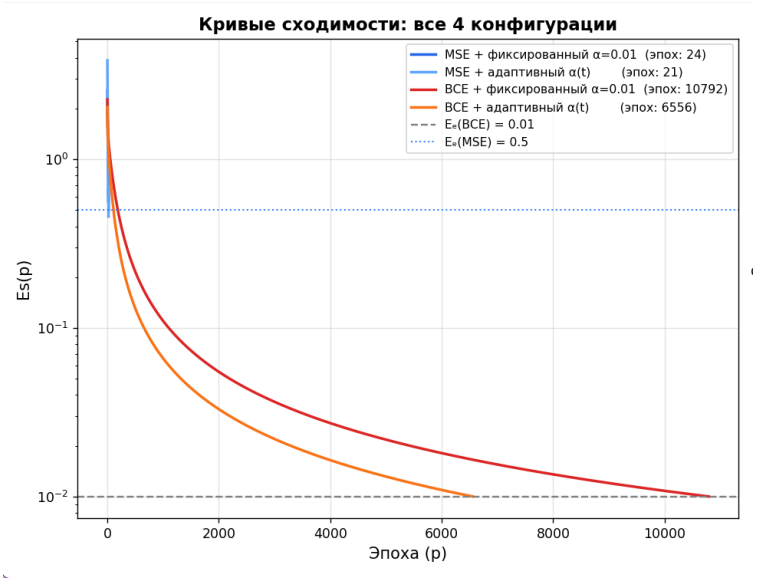
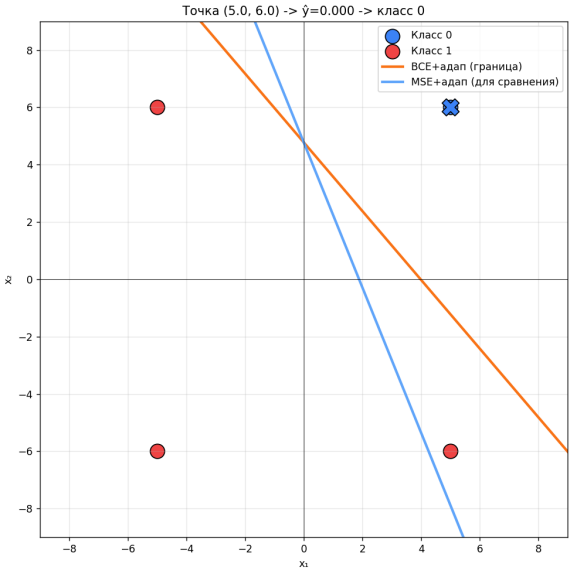


График разделяющих линии для 4 конфигурации:



Вывод: Проделав данную лабораторную работу я изучил принципы бинарной классификации с вероятностной точки зрения. Реализовал однослойную нейронную сеть (логистический регрессор) с сигмоидной функцией активации и функцией потерь в виде бинарной кросс-энтропии (Binary Cross-Entropy, BCE). Провел сравнительный анализ скорости сходимости и качества классификации с моделями, использующими среднеквадратичную ошибку (ЛР №1) и адаптивный шаг обучения (ЛР №2).